

Sistema de Inventario Escolar

CBTa 15

Docente: Mario Alonso Segovia Gutiérrez

Materia: Proyecto de una Plataforma Virtual

Alumno(s): Inurreta Brito Gustavo Enrique, Carlos Alberto Castillo

pinzon, Rodrigo Alejandro Barbosa Vidal y Michael Roman Alavez Colli

Problema que resuelve

La institución necesita llevar un mejor control de los bienes escolares, como computadoras, proyectores, escritorios, mobiliario, herramientas y otros recursos.



Actualmente, el control manual puede provocar pérdida de información, duplicidad de registros, dificultad para saber quién tiene asignado un bien y falta de reportes actualizados.

Objetivo general

Desarrollar un sistema web que permita registrar, consultar, asignar, importar, exportar y dar seguimiento a los bienes escolares de una institución, facilitando la administración del inventario y la toma de decisiones.



Estado actual del proyecto

El sistema ya cuenta con una base funcional. Actualmente permite iniciar sesión, administrar bienes, áreas, personal, asignaciones, historial de movimientos, pendientes, usuarios, reportes y códigos de barras.



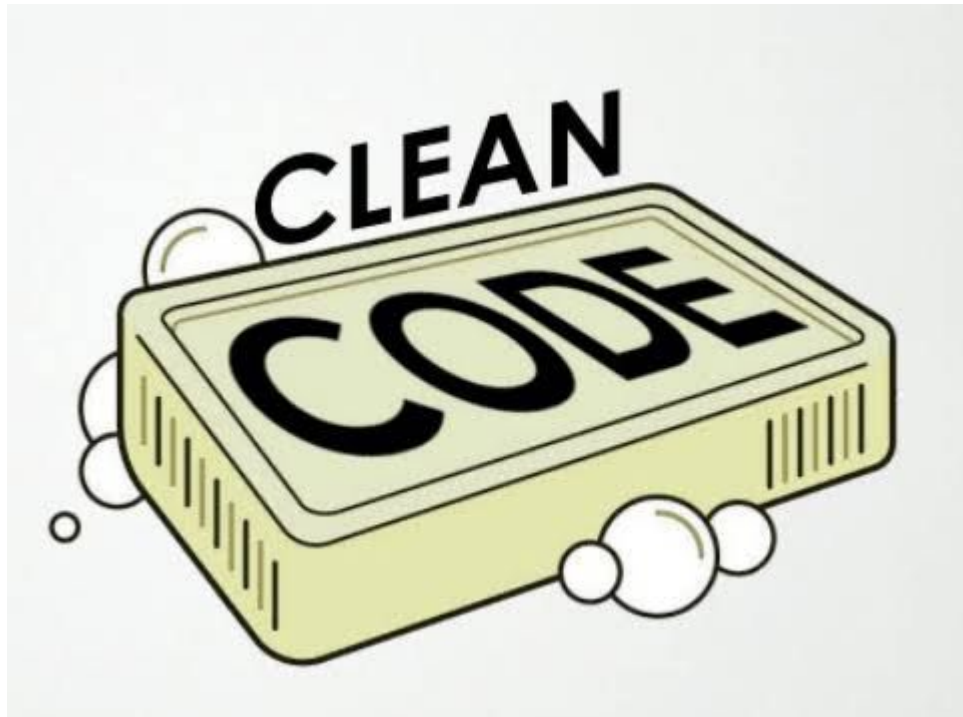
también cuenta con importación de archivos CSV/Excel y exportación de reportes. Sin embargo, todavía existen mejoras pendientes relacionadas con validaciones, seguridad, reportes, pruebas, roles, usabilidad y control de duplicados.

Tecnologías que utilizarán.

1. Laravel.
2. PHP.
3. Blade.
4. MySQL.
5. HTML5.
6. CSS3.
7. JavaScript.
8. Bootstrap.
9. Migraciones de base de datos.
10. Autenticación mediante roles.
11. PHPUnit para pruebas funcionales.
12. GitHub para el control de versiones
13. notion



Reglas de Clean Code



Regla de Clean Code	¿Por qué implementarla?	¿Qué problema evita?	¿Cómo se aplicará?
1. Utilizar nombres claros y descriptivos	Facilita identificar la función de variables, métodos y clases.	Confusión y dificultad para entender el código.	Usar nombres descriptivos y evitar abreviaturas o nombres genéricos.
2. Separar responsabilidades por módulo	Organiza mejor las funciones del sistema.	Controladores muy grandes y difíciles de mantener.	Un controlador por cada módulo (Bienes, Áreas, Personal, Reportes, etc.).
3. Validar los datos antes de guardarlos	Garantiza la integridad de la información.	Registros incorrectos, incompletos o inválidos.	Aplicar validaciones de Laravel y mostrar mensajes claros de error.
4. Evitar la duplicación de código	Facilita el mantenimiento del sistema.	Repetición de lógica y mayor riesgo de errores.	Reutilizar funciones, métodos y servicios comunes.
5. Mantener funciones pequeñas y enfocadas	Hace el código más claro y fácil de probar.	Métodos largos con múltiples responsabilidades.	Cada función realizará una sola tarea específica.
6. Utilizar comentarios solo cuando sean necesarios	Mantiene el código limpio y entendible.	Comentarios innecesarios o desactualizados.	Comentar únicamente procesos complejos o especiales.
7. Utilizar constantes para valores fijos	Mejora la claridad y facilita cambios futuros.	Uso de valores mágicos en el código.	Definir constantes o enumeraciones para estados y roles.
8. Mantener un formato uniforme del código	Facilita la lectura y el trabajo en equipo.	Inconsistencias de formato y organización.	Seguir las convenciones de Laravel/PHP (indentación, nombres y estructura).

APLICACIÓN DE LOS PRINCIPIOS SOLID

Principio 1. Responsabilidad Única (Single Responsibility Principle)

¿En qué consiste?

El principio de Responsabilidad Única establece que una clase, módulo o componente debe tener una sola responsabilidad o un único motivo para cambiar. Esto significa que cada parte del sistema debe encargarse únicamente de una tarea específica, evitando mezclar diferentes procesos dentro de una misma clase.

¿En qué módulo o parte del sistema será aplicado?

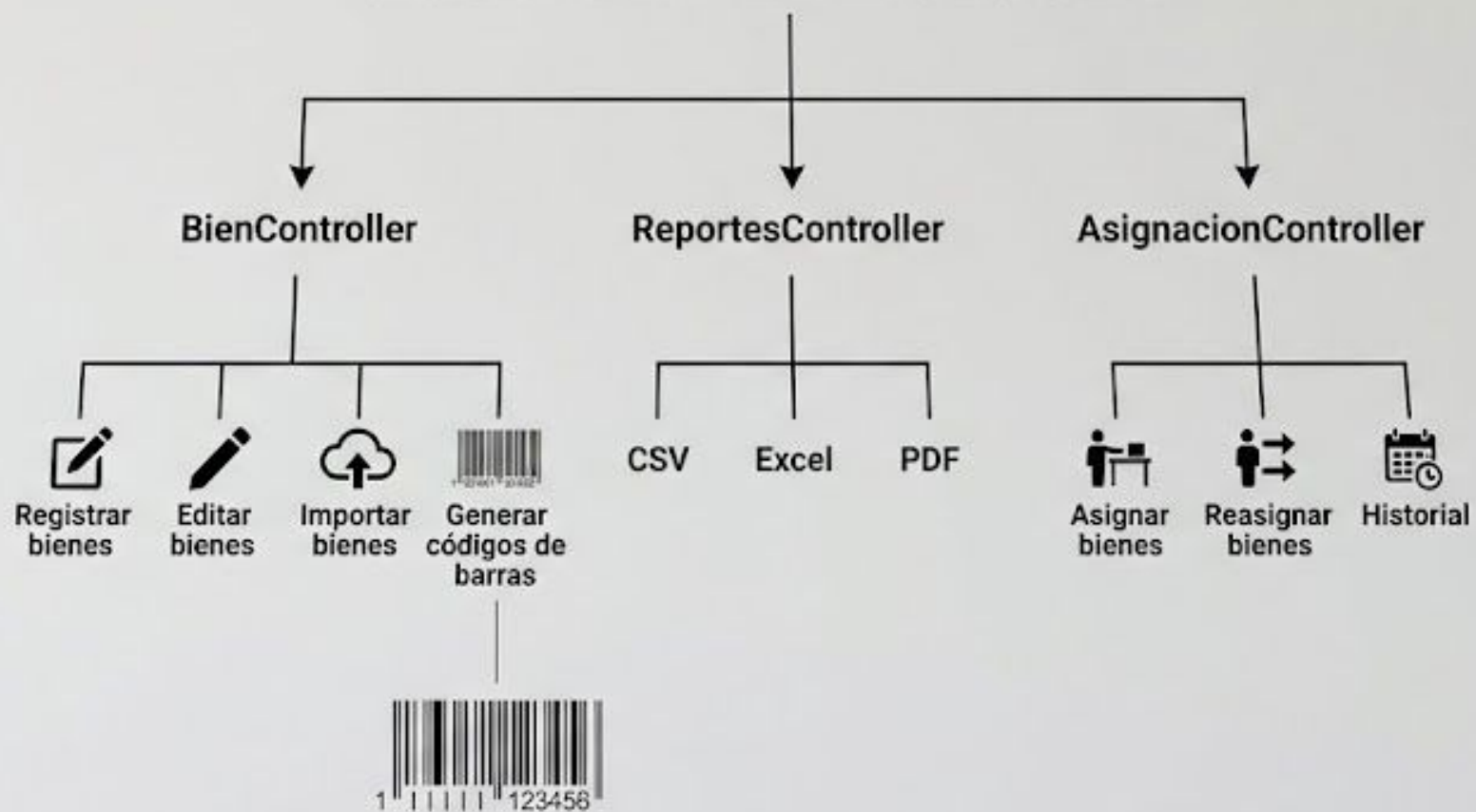
En el Sistema de Inventario Escolar este principio se aplicará principalmente en los controladores del sistema.

Por ejemplo:

- **BienController** será responsable únicamente de la administración de los bienes.
- **ReportesController** será responsable exclusivamente de generar reportes y exportaciones.
- **AsignacionController** administrará únicamente las asignaciones de bienes.
- **PersonalController** administrará el personal.
- **AreaController** administrará las áreas.
- **UsuarioController** administrará los usuarios del sistema.

De esta forma cada controlador tendrá funciones claramente definidas y será más fácil localizar cualquier modificación que deba realizarse.

SISTEMA DE INVENTARIO ESCOLAR



Principio 2. Abierto / Cerrado (Open / Closed Principle)

¿En qué consiste?

Este principio establece que una clase o módulo debe estar abierto para extender su funcionalidad, pero cerrado para modificar el código existente. Esto significa que cuando se requiera agregar una nueva característica, se deberá hacer mediante extensiones o nuevas clases sin alterar el funcionamiento de lo que ya existe.

¿En qué módulo o parte del sistema será aplicado?

Este principio se aplicará principalmente en el módulo de reportes.

Actualmente el sistema permite exportar información en los formatos:

- CSV
- Excel
- PDF

¿Qué problema evitará?

Evitará que al agregar nuevas funcionalidades se modifique código que ya funciona correctamente, disminuyendo la posibilidad de introducir errores o afectar módulos existentes.

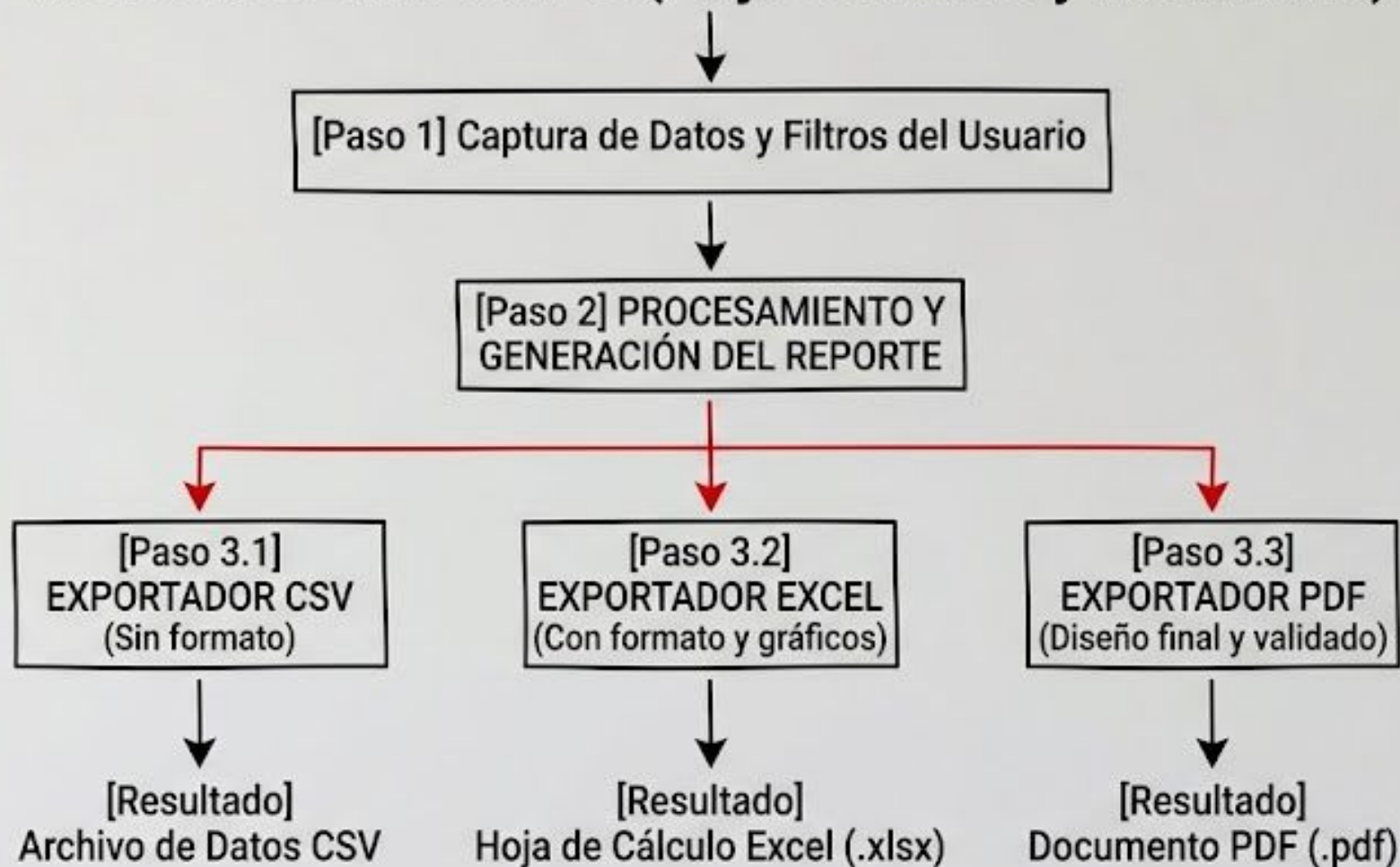
También permitirá que el sistema pueda crecer sin necesidad de reescribir funcionalidades completas.

¿Qué beneficios traerá al mantenimiento del proyecto?

Entre los principales beneficios se encuentran:

- Facilitar el crecimiento del sistema.
- Reducir el riesgo de errores.
- Permitir agregar nuevos formatos de exportación.
- Reutilizar la lógica existente.
- Mantener organizado el módulo de reportes.
- Reducir el tiempo de mantenimiento.

MÓDULO DE REPORTES (Flujo Detallado y Alternativo)



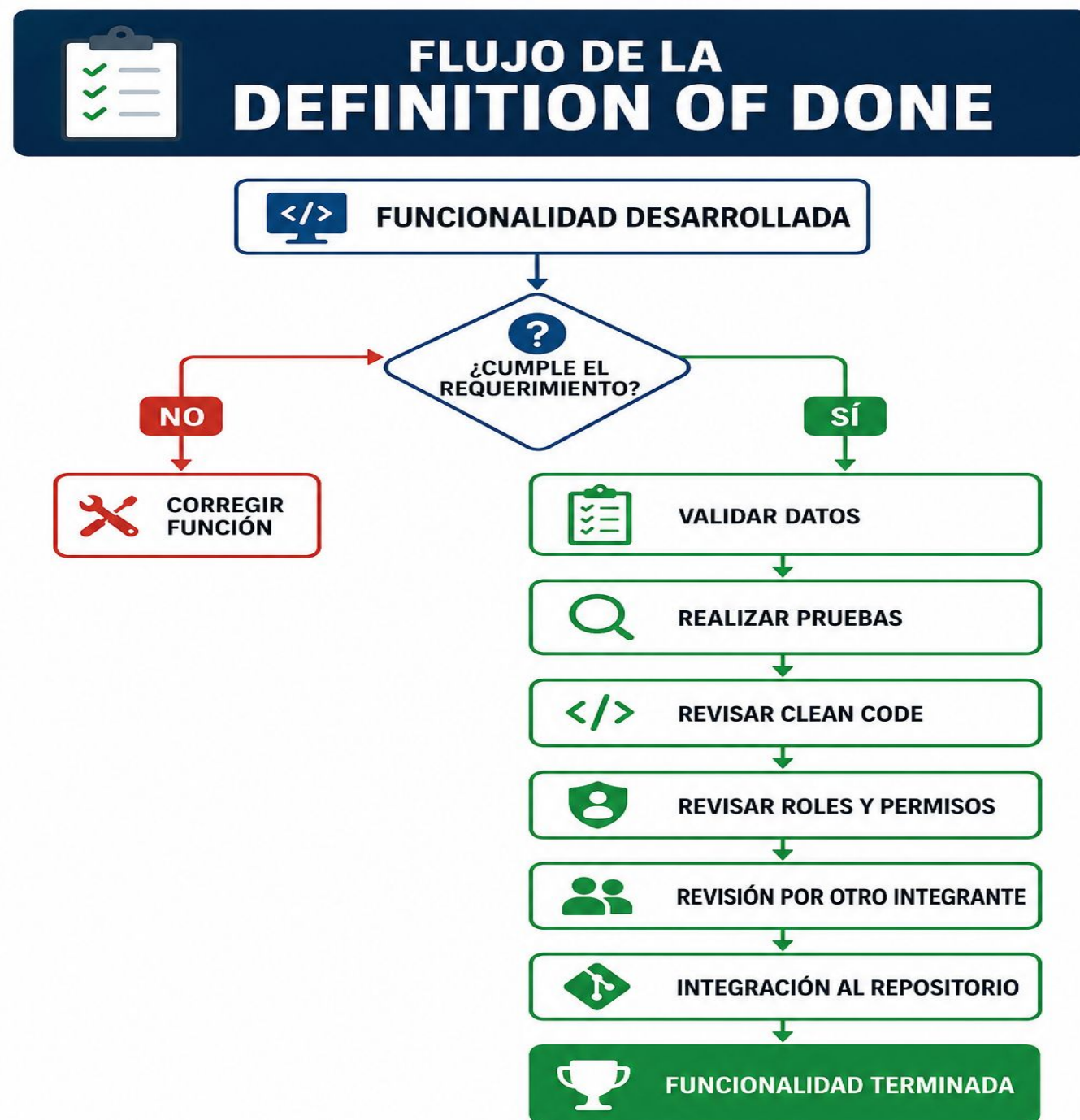
PARTE 4.

DEFINITION OF DONE

La **Definition of Done (DoD)** representa el conjunto de criterios mínimos que una funcionalidad debe cumplir antes de considerarse completamente terminada. Su objetivo es asegurar que todas las funcionalidades desarrolladas mantengan un nivel uniforme de calidad, funcionamiento y documentación, evitando entregar módulos incompletos o con errores. En el Sistema de Inventario Escolar una funcionalidad solamente será considerada terminada cuando cumpla con todos los siguientes criterios.

CRITERIOS DE ACEPTACIÓN			
1		La funcionalidad cumple completamente con el requerimiento solicitado por el cliente o el equipo.	
2		El código fue implementado dentro del módulo correspondiente.	
3		El desarrollo sigue las reglas de Clean Code definidas por el equipo.	
4		Las clases, métodos y variables poseen nombres claros y descriptivos.	
5		Todos los datos ingresados por el usuario son validados correctamente.	
6		Se respetan los permisos establecidos para cada rol del sistema.	
7		La funcionalidad no presenta errores visibles durante su ejecución.	
8		Se realizaron pruebas manuales desde la interfaz del sistema.	
9		Se ejecutaron pruebas funcionales cuando fue necesario.	
10		La nueva funcionalidad no afecta otros módulos existentes.	
11		La información se almacena correctamente en la base de datos.	
12		Las relaciones entre bienes, áreas, personal y usuarios funcionan correctamente.	
13		Los mensajes de éxito, advertencia y error son claros para el usuario.	
14		Los reportes muestran información correcta.	
15		Las exportaciones en CSV, Excel y PDF funcionan correctamente.	
16		El diseño mantiene la misma apariencia que el resto del sistema.	
17		El código fue revisado por otro integrante del equipo.	
18		La funcionalidad fue integrada correctamente al repositorio GitHub.	
19		No existen conflictos pendientes antes de integrar los cambios.	
20		La documentación fue actualizada cuando fue necesario.	
21		El código innecesario fue eliminado.	
22		La funcionalidad cumple con las medidas básicas de seguridad.	
23		Las acciones importantes quedan registradas en el historial cuando corresponde.	
24		La funcionalidad fue probada en diferentes escenarios de uso.	
25		La funcionalidad está lista para ser presentada al cliente.	

Flujo de la Definition of Done



Parte 5: Deuda Técnica

Riesgos Identificados

Decisiones:

- Escribir código desordenado o duplicado.
- No documentar ni probar los cambios.
- Usar valores fijos y omitir revisiones de código.
- Modificar la base de datos o reportes sin un proceso adecuado.
- Aumentar errores y dificultar el mantenimiento del sistema.

Situaciones:

Mala planificación y presión por las entregas.

- Falta de comunicación y buenas prácticas.
- Copiar código sin comprenderlo.
- No realizar pruebas ni validaciones.
- No usar control de versiones ni revisar el trabajo del equipo.

Estrategias :

- Mantener una estructura organizada y código limpio.
- Aplicar principios **Clean Code** y **SOLID**.
- Usar **Git/GitHub** con ramas para controlar cambios.
- Realizar revisiones y pruebas antes de integrar funcionalidades.
- Documentar cambios importantes y eliminar código innecesario.

Funciones Core

- Autenticación de usuario.
- CRUD completo de bienes.
- Reporte básico en PDF.

Futuras Iteraciones

- Generación de códigos de barras.
- Riesgos Identificados
- Gráficos predictivos de uso.
- Dashboard avanzado para admin.

MVP: Funcionalidades Indispensables

Razón: Priorizar el control principal del inventario escolar.

- Incluir funciones esenciales: registro, asignación, consultas y reportes.
- Resolver el problema de organización y control de los bienes.
- Dejar las funciones avanzadas para versiones posteriores.